

C Programming

Trainer : Rajiv K

Email: rajiv.kamune@sunbeaminfo.com



Agenda

- **Width Specifiers**
- **Expressions and Operators**

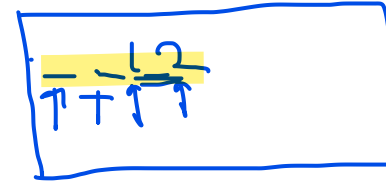


Width Specifiers :

- `int num = 12;`
- `printf("%4d", num);`

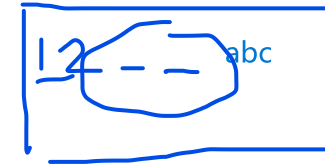
output : `__ 12`

`2 + 2 => 4`



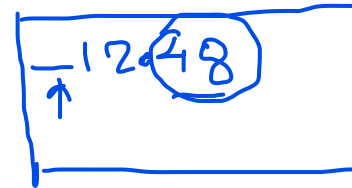
- `int num = 12;`
- `printf("%-4d", num);`

output : `12 __`



- `float fval= 12.48;`
- `printf("%6.2f", fval);`

output : `_ 12.48`



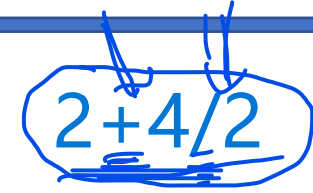
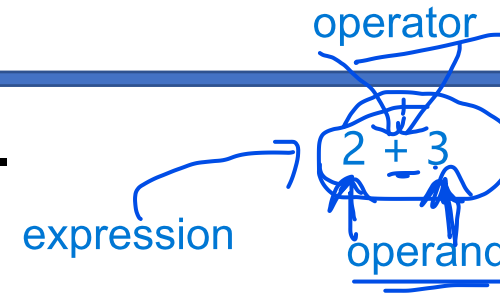
$$5 + 1 = 6.2$$



Expressions and Operators :

- An expression is made up of **operands and operators**.

- Example : `int a = 2 + 3;`
 - Here a, 2, 3 are operands. '+' '=' are operators.



- Precedence of operators :

Each operator in C has a **precedence associated with it**. In a compound expression where there are multiple operators involved, then the expression is solved as per their precedence.

$2 * 4 / 2$

- Associativity :

In a compound expression when several operators are of **same precedence**, then operators are evaluated according to their associativity either left to right or right to left.

- Classification of operators

- **Unary Operators** : Unary Operator operates only on one operand.

Example: expression : `-3` here `-` is a unary minus operator.

types of unary operator are : `&`, `sizeof`, `!` (logical negation), `~` (bitwise negation), `++` (increment), `--` (decrement).

```
int num1=10;
num1++; // num1=10+1
```

- **Binary Operators** : Binary operator operates on 2 operands

Example: expression : `2 - 3`, `-` acts as a binary minus operator as it operates on two operands 2 and 3.

types of binary operators are `*`, `/`, `<<` (left shift), `>>` (Right shift), Logical And(`&`), Bitwise And(`&`)

$10 + 20$

- **Ternary Operator** : A ternary operator operates on three operands

Example : Conditional operator (`? :`) is the only ternary operator in C

```
int num1, num2;
```

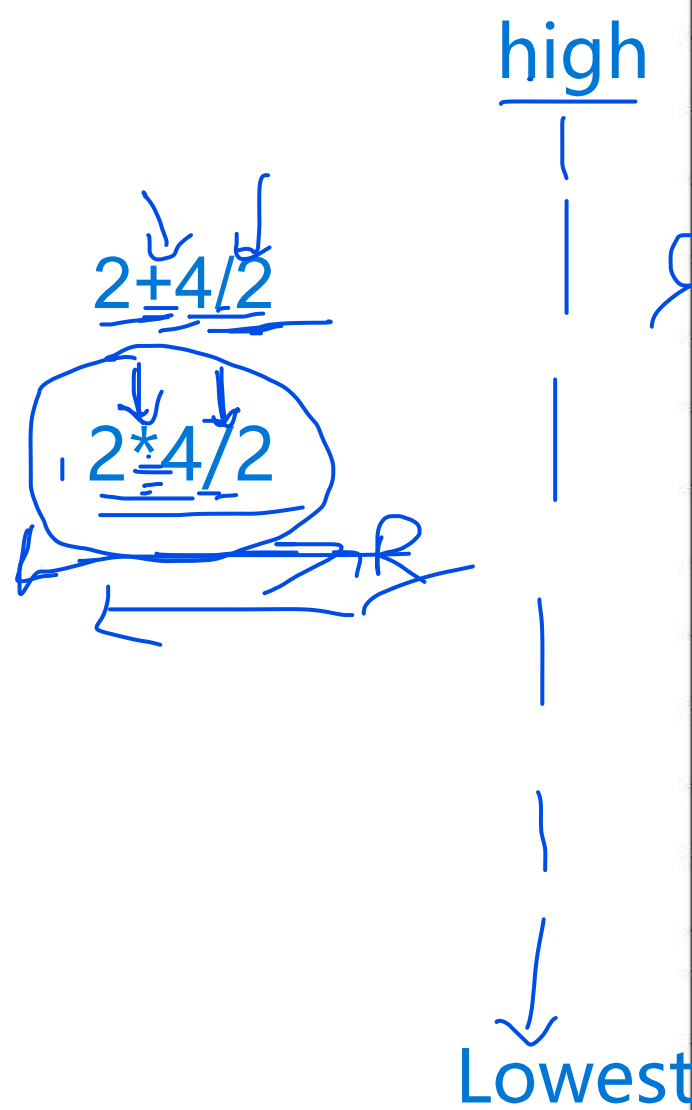


Operators :

- Arithmetic Operators (+ , -, * , / , %)
12%2=> 0
5%2=> 1
- Assignment & shorthand Operators (= , += , -= , *= , /= , %= , &= , |= , ^= , ~= , <<= , >>=)
- Relational Operators (< , <= , > , >= , !=) ==
- Logical Operators (&&, ||, !)
- Conditional Operator (? :)
- Bitwise Operators (& , | , ^ , ~ , << , >>)
- Unary Operators (+ , - , ++ , -- , * , & (address of))
- Special Operator (, comma operator , sizeof() , [])



Operator Precedence and Associativity :



OPERATOR	TYPE	ASSOCIATIVITY
() [] . ->		left-to-right
++ -- + - ! ~ (type) * & sizeof	Unary Operator	right-to-left
* / %	Arithmetic Operator	left-to-right
+ -	Arithmetic Operator	left-to-right
<< >>	Shift Operator	left-to-right
< <= > >=	Relational Operator	left-to-right
== !=	Relational Operator	left-to-right
&	Bitwise AND Operator	left-to-right
^	Bitwise EX-OR Operator	left-to-right
	Bitwise OR Operator	left-to-right
&&	Logical AND Operator	left-to-right
	Logical OR Operator	left-to-right
? :	Ternary Conditional Operator	right-to-left
= += -= *= /= %= &= ^= = <<= >>=	Assignment Operator	right-to-left
,	Comma	left-to-right

Unary operators :

- **Increment operator (++) :**
 - ++ is the increment operator that increments the value of the variable by 1.
 - It is a unary operator and its operand must be a variable. Any expression or constant is not allowed.
 - This operator has 2 types :
 - **Pre-increment :**
 - In this, the value of the variable is incremented first and then the modified value is used.
 - Syntax : ++num;
 - **Post-increment :**
 - In this, the value of the variable is first used and then its modified.
 - Syntax : num++;
- **Decrement operator (--)** :
 - -- is the decrement operator that decrements the value of the variable by 1.
 - Other properties are similar to the increment operator.
 - It has 2 types :
 - **Pre-decrement :**
 - In this, the value of the variable is decremented first and then the modified value is used.
 - Syntax : --num;
 - **Post-decrement :**
 - In this, the value of the variable is first used and then its modified.
 - Syntax : num--;



Bitwise operator AND (&) :

- Bitwise operators can be used on integral types. They operate on binary bits corresponding to that number.
- Bitwise AND & :

A	B	A&B
0	0	0
0	1	0
1	0	0
1	1	1

- Bitwise AND operator on the individual bits of the operand according to the above truth table
- Example : - 10 & 5
- 0000 1010 -> Binary of 10
- 0000 0101 -> Binary of 5
- -----
- 0000 0000 -> O/P is 0



Bitwise operator OR (|) :

- Bitwise OR | :

x	y	x y
1	1	1
1	0	1
0	1	1
0	0	0

- Bitwise OR operators on the individual bits of the operand according to the above truth table.
- Example : -10 | 5
- 0000 1010 -> Binary of 10
- 0000 0101 -> Binary of 5
- -----
- 0000 1111 -> O/P is 15



Bitwise operator XOR (^) :

- Bitwise XOR ^ :

Input		Output
A	B	A xor B
0	0	0
0	1	1
1	0	1
1	1	0

- Bitwise XOR operators on the individual bits of the operand according to the truth table shown above
- Example : $-10 \wedge 13$
- 0000 1010 -> Binary of 10
- 0000 1101 -> Binary of 13
- -----
- 0000 0111 -> O/P is 7



Bitwise operator Not (~) :

- **Bitwise NOT ~ :**

- Bitwise NOT operator results in one's complement of its operand.
- Each bit is inverted i.e. 0 to 1 and 1 to 0.

NOT ~	
INPUT	OUTPUT
0	1
1	0



Shift operators :

- **Left shift operator :**

- $x \ll n$: shifts the value of x left by n bits
- $13 \ll 1$
- $13 \ll 1 = 26$

- **Formula :** $\text{num} \ll n = \text{num} * 2^{\text{raise to } n}$

- $13 \ll 1 = 13 * 2^1$
 - $= 13 * 2 = 26$

- **Right shift operator :**

- $x \gg n$: shifts the value of x right by n bits
- $13 \gg 1$
- $13 \gg 1 = 6$

- **Formula :** $\text{num} \gg n = \text{num} / 2^{\text{raise to } n}$

- $13 \gg 1 = 13 / 2^1$
 - $= 13 / 2 = 6$



Points to remember :

- If **precedence** of two operators in an expression is **same**, their **associativity** is considered to decide their binding with operands.
- **Data type conversions and ranges** should be considered while doing arithmetic operations.
- **sizeof() is compile time operator**. Expressions within sizeof() are not executed at runtime.
- **Relational and logical operators** always result in 0 or 1.
- In logical AND, if first condition is false, second condition is not evaluated. Result is false.
- In logical OR, if first condition is true, second condition is not evaluated. Result is true.
- Increment/Decrement operators in arithmetic expressions are compiler dependent.



Combination:

- unary ops with logical ops
 - `int num1 = 1, num2 = 30, num3 = 35;`
 - `int res = num1++ || ++num2 && ++num3;`
- unary ops with comma and bracket ops
 - `int num1 = 10, num2 = 20;`
 - `res = num1++, ++num1, num1++;`
 - `int result = (++num2, num2++, ++num2);`



Thank You

